

ALCHEMIX

Phase 2 Analytical Co-Pilot

Business Analysis, Solution Architecture, and Implementation Roadmap

Project: AlchemiX - LAUNCH26 Relic Ring Protocol

System: Zeta-26

Document status: Pre-implementation planning complete

Prepared: 06 July 2026

IMPLEMENTATION HOLD

No source-code implementation begins until the exact phrase

RELEASE THE KRAKEN

Prepared from the project handoff, Phase 2 challenge brief, universe configuration, and supplied historical datasets.

Contents

Executive Summary

1. Current Project State

- 1.1 What has been completed
- 1.2 Existing technical baseline
- 1.3 Current known gaps that remain relevant
- 1.4 Phase 1 assets that must remain unchanged

2. Phase 2 Challenge Interpretation

- 2.1 Business problem
- 2.2 Required agent behavior
- 2.3 Mandatory output contract
- 2.4 Live API expectations
- 2.5 Evaluation-aligned success criteria

3. Dataset Assessment and Initial Findings

- 3.1 Dataset inventory
- 3.2 Data-quality findings
- 3.3 Congestion pattern
- 3.4 Trust pattern
- 3.5 Targeting pattern
- 3.6 Data governance rules

4. Recommended Solution Architecture

- 4.1 Architectural principle
- 4.2 High-level component model
- 4.3 Component responsibilities

5. Technology Decisions and Alternatives

- 5.1 Technology decision table
- 5.2 Why FastAPI is used instead of embedding models directly in Go
- 5.3 Why a cloud LLM is not the default agent brain

6. Analytical Model Design

- 6.1 Congestion model
- 6.2 Trust model
- 6.3 Targeting-risk model
- 6.4 Unseen-vector handling

7. Sequential Routing Workflow

- 7.1 End-to-end workflow
- 7.2 Important interpretation
- 7.3 Safety guards

8. Proposed Code Structure

9. Environment Setup and Installation Order

- 9.1 Prerequisites
- 9.2 First commands after repository access

1. Create a Phase 2 branch

2. Confirm the current Go application builds

3. Confirm the frontend builds

4. Confirm Wails development mode

5. Confirm production build

6. Inspect current Docker setup

- 9.3 Create the analytics environment

Windows PowerShell

macOS/Linux

- 9.4 Environment variables

10. Implementation Roadmap

Phase 0 - Repository Baseline and Gap Confirmation

Phase 1 - Data Validation and Reproducible Training Foundation

Phase 2 - Train the Three Analytical Models

Phase 3 - Analytical Tool Service

Phase 4 - Live Telemetry Integration

Phase 5 - Natural-Language Request Parser

Phase 6 - Sequential Co-Pilot and True-Cost Routing

Phase 7 - UI and Explainability

Phase 8 - Resilience, Chaos, and Generalization Testing

Phase 9 - Packaging and Final Runbook

11. Testing Strategy

11.1 Unit tests

11.2 Model tests

11.3 Integration tests

11.4 Acceptance scenarios

12. Non-Functional Requirements

13. Risks and Mitigations

14. Requirements Traceability Matrix

15. Definition of Done

16. Recommended First Work Session After Release

17. Source Material

Release Gate

Executive Summary

Phase 1 created a working Go + Wails desktop application that can load the Zeta-26 universe dynamically, calculate physically valid routes, enforce the maximum void-hop distance, calculate latency, translate payloads between planetary codices, record ordered hop logs, simulate failures, and visualize the network in React.

Phase 2 does not replace that work. It adds an **Analytical Co-Pilot Agent** in front of the Phase 1 router. The agent must accept a natural-language request, extract the origin, destination, and payload, generate the original physics route, obtain live link state from the Chimera API, evaluate congestion, telemetry trust, and targeting risk, calculate a dynamic True Cost, and reconsider the route at every hop.

The recommended implementation keeps the existing Go/Wails/React system as the control plane and adds a small Python analytics sidecar. Python is used only for model training and prediction because the supplied data and model requirements are best served by pandas and scikit-learn. Go remains responsible for routing, sequential agent orchestration, live API integration, safety rules, fallback behavior, output conformance, and decision logging. This minimizes change to working Phase 1 logic and avoids a risky rewrite.

The design intentionally avoids a cloud LLM or a large agent framework. The domain has six known planet names and a narrow request structure, so a deterministic natural-language parser is faster, auditable, offline, and easier to test. The system is still agentic because it plans, invokes specialized tools, observes results, changes route decisions, and repeats the loop until delivery.

1. Current Project State

1.1 What has been completed

According to the project handoff, AlchemiX is already a Go + Wails desktop application with a React, TypeScript, and Vite frontend. The following capabilities are completed or mostly completed:

- Dynamic loading of planet and universe metadata from `universe-config.json`.
- Construction of an active routing graph without hardcoding planet values.
- Physics-aware route selection with direct and multi-hop support.
- Enforcement of `max_void_hop_distance_km`.
- Internal planetary fiber, tower processing, atmospheric, and void latency calculations.
- Translation of message text through ASCII and planet-specific codices.
- Ordered packet hop logging.
- Planet, link, and individual tower failure controls.
- Reset and restoration of network state.
- Wails methods and generated frontend bindings.
- React topology view, transmission controls, live event stream, latency panel, hop log, and conversion-history design.
- A centered tower inspector with active and inactive tower states.
- Docker and Docker Compose intent for multi-service deployment.

1.2 Existing technical baseline

Layer	Current technology	Current responsibility
Desktop shell	Wails v2	Connects Go backend and web frontend
Frontend	React + TypeScript + Vite	Controls, topology, results, failure visualization
Backend	Go	Configuration, physics, routing, packets, resilience
Data/config	JSON	Universe metadata, planets, links, capacities
Runtime events	Wails events	Live network updates to the UI
Infrastructure	Docker Compose	Intended service orchestration and repeatable startup

1.3 Current known gaps that remain relevant

The handoff identifies several Phase 1 verification items that should be resolved before or during Phase 2 integration:

1. The exact Go hop-log response schema must be confirmed against a real `StartTransmission` response.
2. TypeScript interfaces still require alignment with the actual backend fields.
3. Conversion history should use backend-generated values rather than frontend fallbacks whenever possible.
4. Docker service names, ports, and startup flow must be verified from the real repository.
5. End-to-end Go, TypeScript, Wails, and Docker builds must be re-run after integration.
6. The current repository itself was not included with this planning request. This report is therefore based on the handoff document, the Phase 2 brief, the configuration, and the datasets. The first implementation task must be a repository inspection and build baseline.

1.4 Phase 1 assets that must remain unchanged

The Phase 2 challenge explicitly states that planet-level physics, configuration schemas, tower rings, fiber-arc transit, codex behavior, and baseline void latency carry over. Therefore:

- The current physics router remains the baseline route generator.
- Existing `Lmax` checks remain mandatory.
- Existing planet, tower, and link failure rules remain active.
- The new agent may change edge costs or choose a detour, but it must not bypass physical validity.
- The backend remains the source of truth for route execution and delivery evidence.

2. Phase 2 Challenge Interpretation

2.1 Business problem

The network is no longer affected only by physics. The Phase 2 brief describes three forms of link-level risk (challenge brief, pp. 2-3):

7. **Congestion sabotage:** artificial delay increases as load rises, and a highly loaded link may become saturated and unusable.
8. **Spoofed telemetry:** some links report latency values that are consistently better than reality.
9. **Predictable-route targeting:** heavily used links are more likely to be jammed, so always selecting the same apparently optimal route creates risk.

The problem is specifically located on **interplanetary links**. Planetary tower rings and internal transit remain secure.

2.2 Required agent behavior

The Phase 2 solution must (challenge brief, pp. 3-4):

- Accept an unstructured natural-language request.
- Extract **origin**, **destination**, and **message payload** from that text.
- Generate the Phase 1 physics route.
- Operate as a sequential agent rather than making one static decision.
- Pause at each current node.
- Invoke three analytical tools for relevant candidate links:
 - congestion prediction,
 - telemetry trust scoring,
 - targeting-risk prediction.
- Combine these outputs with physical latency into a dynamic True Cost.
- Select or override the next hop.
- Re-evaluate after every committed hop using a new or current live-state snapshot.
- Handle a severed link without dropping the packet or violating L_{max} .
- Handle unseen events safely, including explicit uncertainty or conservative fallback.
- Produce the exact required routing-decision JSON schema.
- Explain every link decision in a way that can be defended during a live audit.

2.3 Mandatory output contract

Every completed routing decision must contain these fields without renaming or omission (challenge brief, p. 5):

```
{
  "origin_id": "Caelum",
  "destination_id": "Aegis",
  "chosen_path": ["Caelum", "Elysium", "Aegis"],
  "link_evaluations": [
    {
      "link_id": "Caelum-Elysium",
      "predicted_congestion_penalty_ms": 12.4,
      "trust_score": 0.91,
      "targeting_risk_score": 0.18,
      "combined_cost": 44.2
    }
  ],
  "final_latency_estimate_ms": 88.7,
  "explanation": "Human-readable decision explanation."
}
```

`link_id` must always contain the two connected planet IDs in alphabetical order.

2.4 Live API expectations

The live service contract is defined on pp. 7-10 of the challenge brief and exposes:

- `GET /` for service information.
- `GET /links` for link topology and capacities.
- `GET /state` for current link conditions.

`GET /state` requires the header:

X-Team-Key: <your-team-key>

The agent consumes the following live fields:

- link_id
- planet_a
- planet_b
- capacity_units
- current_load
- load_ratio
- self_reported_latency_ms
- traffic_share
- status

A link with status = "saturated" is unavailable. A null self-reported latency on a saturated link must never be interpreted as zero.

During build days the response shape is valid but values are intentionally scrambled. Historical CSV files must be used to train final models. The live /state feed is for integration testing during build days and real inference on the final day.

2.5 Evaluation-aligned success criteria

Evaluation area	Required behavior
Congestion accuracy	Predict unseen congestion delay with acceptable error and block saturated links
Trust accuracy	Identify spoofed telemetry without penalizing normal noisy links
Live resilience	Recover from a severed selected link without crash or packet loss
Generalization	Detect out-of-distribution input and apply a safe fallback
Explanation quality	Produce a readable reason for each score and route change
Code and docs	Dynamic configuration, readable modules, repeatable startup, exact schema

3. Dataset Assessment and Initial Findings

3.1 Dataset inventory

The supplied dataset guide defines the role of each file. Local profiling confirms that each CSV contains 6,000 records: 12 interplanetary links across 500 ticks.

Dataset	Purpose	Rows	Primary target
link_traffic_history.csv	Congestion learning	6,000	observed latency and saturated status
link_telemetry.csv	Trust learning	6,000	measured vs self-reported latency discrepancy
link_incident_history.csv	Targeting-risk learning	6,000	jammed_flag

3.2 Data-quality findings

Field	Missing values	Treatment
load_units	203	Do not use when load_ratio is available; validate consistency where both exist
observed_latency_ms	257	Exclude from regression target; preserve saturated records as hard-failure evidence

Field	Missing values	Treatment
self_reported_latency_ms	256	Treat as unavailable; never impute zero
traffic_share	252	Exclude from training rows or use a conservative prior at inference

Only three historical traffic rows are marked saturated. This is too little evidence for a reliable learned saturation classifier. The safer design is:

10. Treat live `status = saturated` as authoritative.
11. Validate the supplied default saturation threshold near `load_ratio >= 0.90` as a secondary guard.
12. Train congestion severity only on usable, non-saturated records.

3.3 Congestion pattern

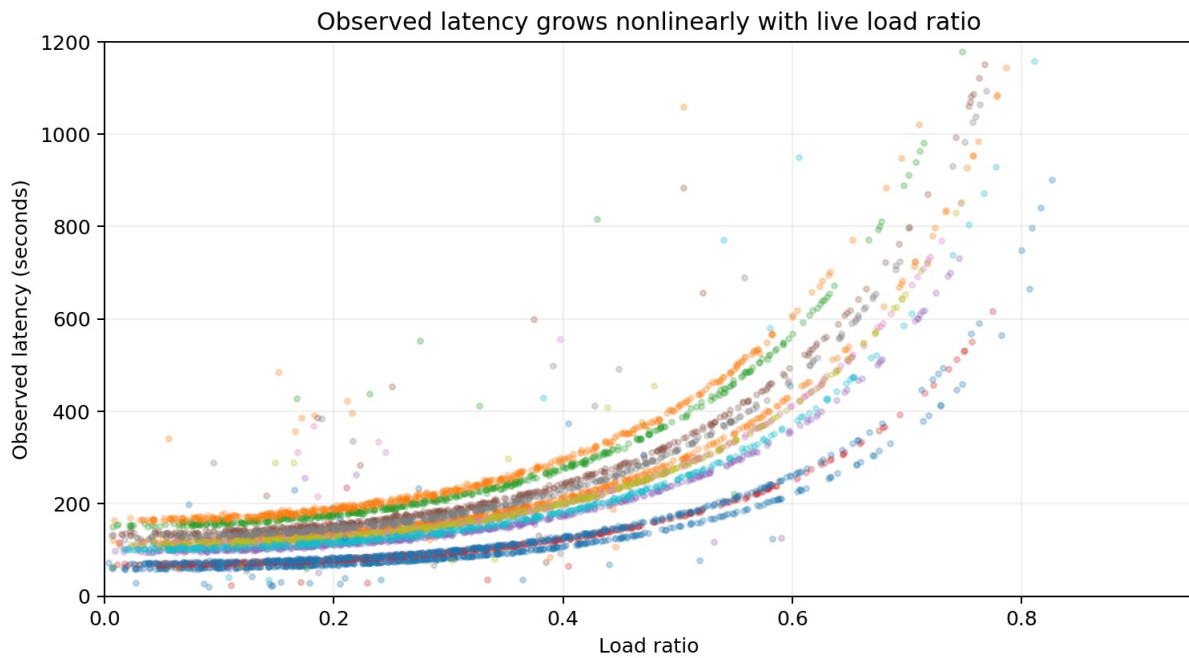


Figure 2. Historical load ratio and observed link latency.

Observed latency has a strong nonlinear relationship with load ratio across most links. A preliminary time-based benchmark using ticks 0-399 for training and 400-499 for testing produced the following architecture-validation results:

Candidate	Test MAE	Test RMSE	Test R-squared	Interpretation
Polynomial Ridge	about 24,175 ms	about 51,377 ms	0.895	Very explainable but less accurate
Gradient Boosting	about 11,861 ms	about 52,232 ms	0.892	Lowest median-like absolute error
Random Forest	about 12,448 ms	about 44,346 ms	0.922	Best overall variance and large-error control

These are exploratory results, not final model-selection evidence. The final implementation should reproduce them through versioned training scripts and cross-validation.

3.4 Trust pattern

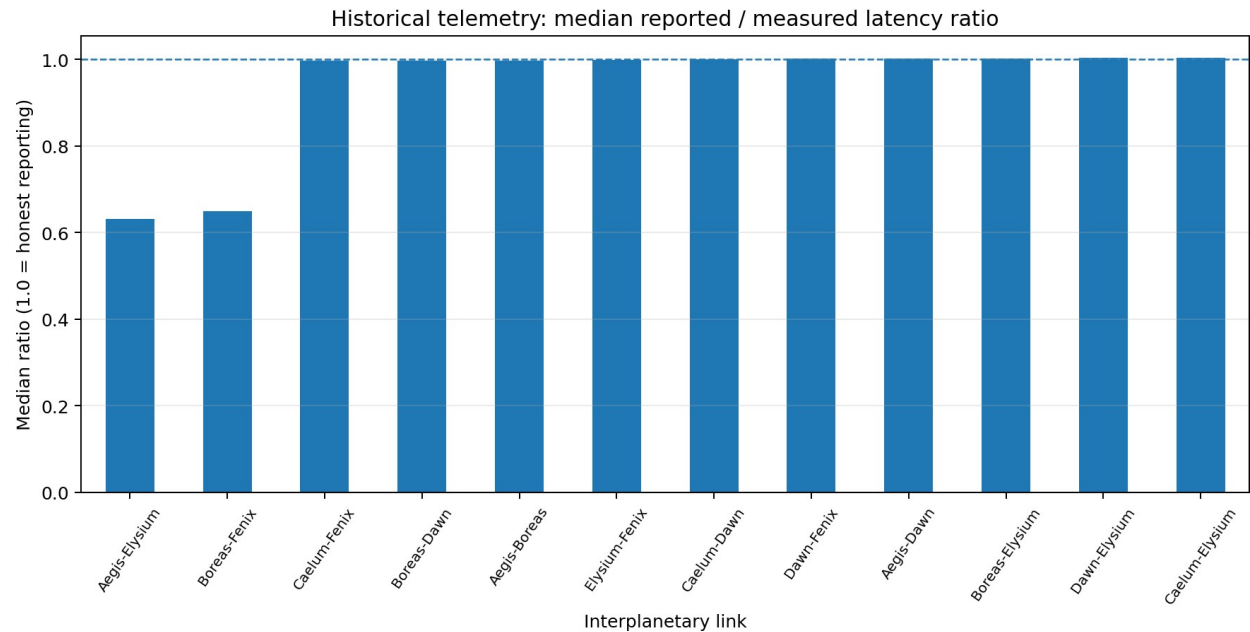


Figure 3. Median self-reported to measured latency ratio by link.

Two links are clearly suspicious in the historical telemetry:

Link	Median reported/measured ratio	Share of valid records below 0.80	Initial interpretation
Aegis-Elysium	about 0.632	about 71.8%	Strong systematic under-reporting
Boreas-Fenix	about 0.649	about 69.2%	Strong systematic under-reporting
Most other links	approximately 1.00	usually below 2%	Normal reporting noise

This separation supports a robust, explainable trust model based on historical under-reporting behavior, uncertainty, and live plausibility checks.

3.5 Targeting pattern

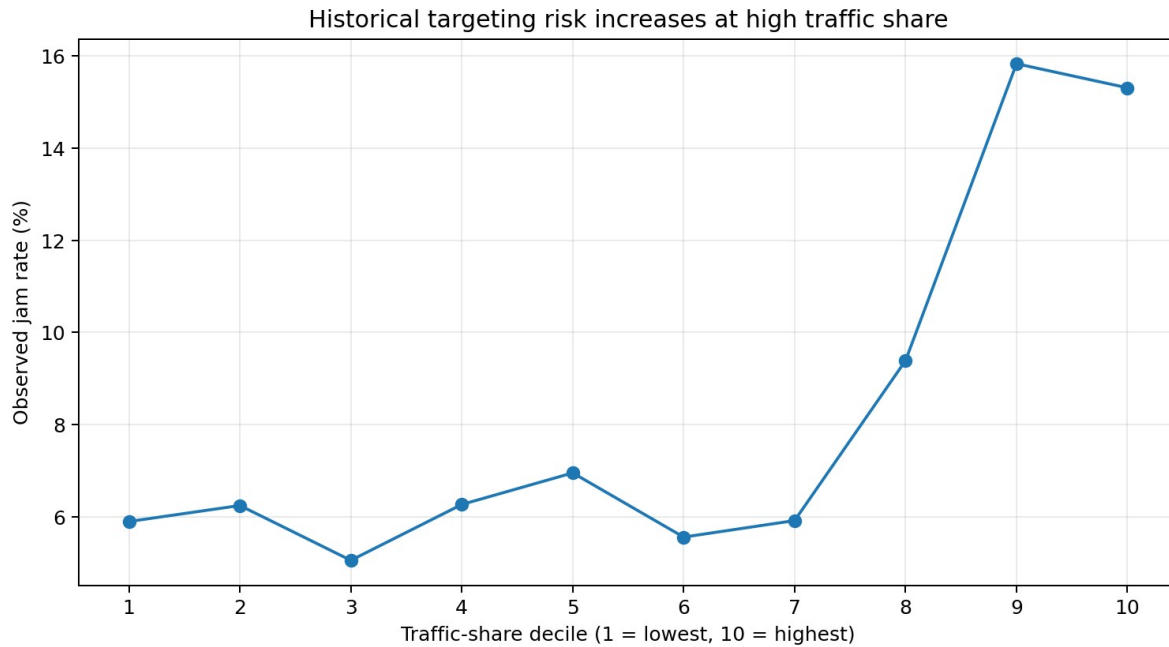


Figure 4. Observed jam rate by traffic-share decile.

Historical jam probability increases materially in the top traffic-share deciles. Low and middle deciles generally show jam rates around 5-7%, while the two highest deciles are around 15-16%. This supports `traffic_share` as the main live feature, with link-specific historical priors used as secondary information.

3.6 Data governance rules

- Historical CSVs are immutable training inputs.
- Live `/state` data must not be appended to the training set during build days.
- Training and evaluation splits must be time-based, not random row splits, to reduce leakage across ticks.
- Every model artifact must have metadata containing training date, source-file hash, features, target, metrics, and model version.
- Missing or out-of-range live values must raise uncertainty and activate a conservative decision path.

4. Recommended Solution Architecture

4.1 Architectural principle

The implementation should use a **hybrid architecture**:

- Go remains the deterministic control plane.
- Python becomes the analytical prediction plane.
- React remains the visual evidence plane.

This preserves the strongest Phase 1 assets while adding the minimum new surface needed for Phase 2.

4.2 High-level component model

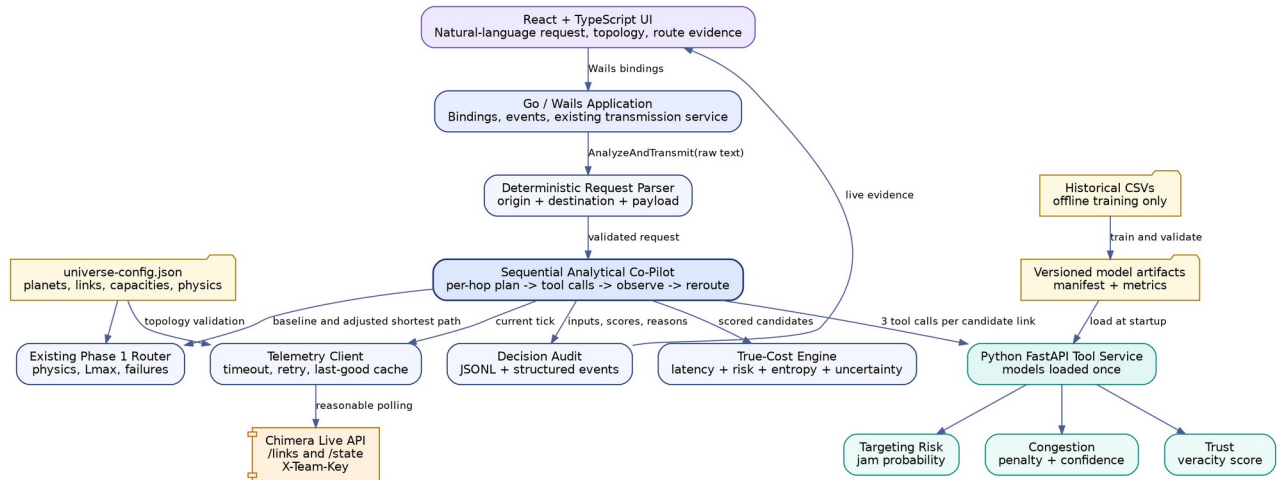


Figure 1. Recommended Phase 2 component architecture.

4.3 Component responsibilities

React + TypeScript UI

- Provide one primary natural-language request box.
- Show parsed origin, destination, and payload for confirmation.
- Display baseline route and chosen adaptive route separately.
- Show current tick and live link status.
- Show congestion penalty, trust score, targeting risk, combined cost, and confidence per evaluated link.
- Show reroute reasons and uncertainty warnings.
- Preserve the existing topology, failure controls, latency panels, conversion history, and hop log.

Go/Wails application

- Expose a new method such as `AnalyzeAndTransmit(rawRequest string)`.
- Coordinate UI state and backend events.
- Keep existing Phase 1 methods available for regression testing.
- Start or check the analytics sidecar during development, or connect to it through Docker Compose.

Natural-language request parser

- Extract exactly one origin and one destination from the known planet dictionary.
- Extract the remaining message payload using a controlled grammar and common separators such as `send`, `from`, `to`, `message`, quotes, or colon.
- Reject ambiguous requests with a helpful error instead of guessing.
- Return a typed object with confidence and validation errors.

Sequential Co-Pilot agent

- Generate the baseline physics route.
- Maintain current node, destination, visited nodes, hop count, and route history.

- Fetch live state at a reasonable interval.
- Evaluate candidate links through the three analytical tools.
- Calculate True Cost and uncertainty.
- Re-run shortest path from the current node with adjusted weights.
- Commit only the first hop, then repeat.
- Prevent loops using visited-node penalties and a maximum-hop guard.
- Produce the exact unified output schema.

Telemetry client

- Read CHIMERA_BASE_URL and CHIMERA_TEAM_KEY from environment variables.
- Apply a short connection timeout and bounded retry policy.
- Cache the last valid snapshot with timestamp and tick.
- Validate /links against universe-config.json during startup.
- Normalize all link IDs alphabetically.
- Treat saturated links as unavailable before model invocation.

Python analytical sidcar

Provide small, typed HTTP tools:

- POST /tools/congestion
- POST /tools/trust
- POST /tools/targeting
- GET /health
- GET /model-info

The service loads model artifacts once at startup. It must not train during live inference.

True-Cost engine

Use a transparent ms-equivalent selection score:

```
combined_cost =
    baseline_physics_ms
    + predicted_congestion_penalty_ms
    + trust_penalty_ms
    + targeting_penalty_ms
    + route_reuse_penalty_ms
    + uncertainty_penalty_ms
```

Recommended conversions:

```
trust_penalty_ms = TRUST_BUDGET_MS * (1 - trust_score)
targeting_penalty_ms = TARGET_BUDGET_MS * targeting_risk_score
route_reuse_penalty_ms = ENTROPY_BUDGET_MS * recent_usage_score
uncertainty_penalty_ms = UNCERTAINTY_BUDGET_MS * uncertainty_score
```

The weights must be configuration values, not hidden constants.

final_latency_estimate_ms should represent expected physical delivery time:

```
sum(baseline physical latency + predicted congestion penalty)
```

It should not include trust, targeting, route-reuse, or uncertainty penalties because those are route-selection costs, not literal latency.

Decision audit

Every candidate evaluation should produce an internal record containing:

- tick,
- current node,
- candidate link,
- baseline physical cost,
- raw model inputs,
- model versions,
- three model outputs,
- confidence or uncertainty,

- each penalty component,
- combined cost,
- selected or rejected outcome,
- human-readable reason.

The exact competition output remains unchanged, while the richer internal audit supports the judge's decision audit.

5. Technology Decisions and Alternatives

5.1 Technology decision table

Decision	Selected technology	Why selected	Alternatives not selected
Preserve desktop	Go + Wails	Existing working backend and bindings; low rewrite risk	Electron would duplicate the shell and increase package size; Python GUI would discard Phase 1 work
Preserve frontend	React + TypeScript + Vite	Existing UI, strong typing, fast component updates	Rewriting in another UI framework adds no scoring value
Agent control plane	Go packages	Deterministic, concurrent, close to existing router and failure state	LangChain-style orchestration adds abstraction and runtime complexity
Model training	Python + pandas + scikit-learn	Fast data preparation, mature tabular ML, reproducible metrics	Go ML libraries have a smaller ecosystem; deep learning is unnecessary for 6,000-row tables
Prediction integration	Local FastAPI sidecar	Models stay in Python; stable typed HTTP boundary; loaded once	Running a Python process per prediction is slow; gRPC is extra setup; ONNX can be a later optimization
Data validation	Pydantic models plus explicit pandas checks	Simple, typed request/response validation	A full data-quality platform is excessive for a three-day challenge
Model storage	Versioned local artifacts plus metadata JSON	Offline, fast, easy to package and inspect	Cloud model registry creates internet and credential dependency; MLflow is useful but heavy here
Audit storage	Append-only JSONL and in-memory ring buffer	Human-readable, easy to stream to UI, no database migration	SQLite is valid but not necessary for the small live audit volume
Natural-language parsing	Deterministic domain parser	Closed planet vocabulary, predictable output, no internet	Cloud LLMs add latency, nondeterminism, cost, and JSON-format risk; local LLMs require unnecessary resources
Deployment	Docker Compose for analytics and supporting services	Repeatable build and health checks; matches project intent	Kubernetes is far beyond the scale; manual environment setup is error-prone
Logging	Go structured logging and Python structured JSON logs	Searchable correlation IDs and consistent audit	Plain console text is hard to correlate across the two runtimes

5.2 Why FastAPI is used instead of embedding models directly in Go

A direct Go-only inference design would produce one runtime, but training and converting tree models into a Go-compatible format would consume valuable development time. Native ONNX Runtime also adds platform-specific binaries and packaging risk inside a Wails desktop application. A local FastAPI process keeps the model code in the same ecosystem used for training and creates an explicit tool-call boundary that maps cleanly to the challenge language.

The Go agent must include a fallback when the sidecar is unhealthy. This prevents the sidecar from becoming a single point of failure.

5.3 Why a cloud LLM is not the default agent brain

The challenge needs unstructured request parsing and tool orchestration, but it does not require generative text reasoning. There are only six known planets and a simple payload extraction task. A deterministic parser provides:

- stable results,
- no API key or internet dependency,
- lower latency,
- easy unit testing,
- clear error handling,
- no prompt-injection exposure,
- no accidental changes to the required JSON schema.

An optional parser interface can allow an LLM fallback later, but it should remain disabled for the live demo unless judges explicitly require it.

6. Analytical Model Design

6.1 Congestion model

Objective

Predict the additional delay caused by Chimera for a non-saturated link.

Inputs

- normalized `link_id`,
- `load_ratio`,
- `capacity_units`,
- optionally baseline physical link latency from Phase 1,
- derived pressure features such as `load_ratio^2` and `load_ratio / (1 - load_ratio)`.

Target

Prefer a penalty target:

```
congestion_penalty_ms = max(0, observed_latency_ms - baseline_physics_ms)
```

If exact baseline physical latency cannot be aligned confidently, predict observed latency and subtract baseline only at inference. The chosen definition must be documented and used consistently.

Recommended champion and fallback

- Champion candidate: Random Forest or Gradient Boosting selected by time-based validation.
- Explainable fallback: polynomial Ridge regression.
- Hard rule: saturated status always returns unavailable, not a numeric penalty.

Output

```
{
  "predicted_congestion_penalty_ms": 12450.0,
  "confidence": 0.87,
  "is_out_of_distribution": false,
  "reason": "High load ratio relative to historical values for this link."
}
```

6.2 Trust model

Objective

Estimate how believable the link's self-reported latency is.

Training approach

Use robust link-level statistics:

- median reported/measured ratio,
- median absolute percentage error,
- frequency of severe under-reporting,
- sample count,
- dispersion and missing-report rate.

At live inference, combine the historical link prior with current plausibility:

- Is self-reported latency below the physical lower bound?
- Is it implausibly lower than the congestion model's expected latency?
- Is the value null while status is not saturated?
- Is the live value far outside the link's historical distribution?

Output scale

- 1.0 means highly trustworthy.
- 0.0 means strongly suspected spoofing.

The initial evidence indicates that *Aegis-Elysium* and *Boreas-Fenix* should receive materially lower historical priors than other links.

6.3 Targeting-risk model

Objective

Predict the probability that a link will be jammed in the current condition.

Inputs

- `traffic_share`,
- `normalized_link_id`,
- link historical jam prior,
- optional recent route-reuse score maintained by AlchemiX.

Recommended model

A calibrated gradient-boosting classifier or logistic regression benchmarked with time-based validation. The final choice should prioritize probability calibration and Brier score because the output is a risk probability, not only a yes/no classification.

Route entropy

The model's traffic-share risk should be supplemented by an AlchemiX-specific route-reuse penalty. Maintain an exponentially decayed count of recently selected links. Repeatedly selecting the same link increases the True Cost even when it remains physically fast.

This creates controlled diversity without random unsafe routing.

6.4 Unseen-vector handling

For every tool, calculate an uncertainty indicator from rules such as:

- input outside historical min/max,
- missing required field,
- unknown link,
- sidecar model version mismatch,
- extreme disagreement between self-report and expected latency,
- all available candidate links receiving low confidence.

When uncertainty is high:

13. apply an uncertainty penalty,
14. prefer a physically valid link with higher trust and lower utilization,
15. `log_out_of_distribution = true`,
16. state the uncertainty in the explanation,
17. return undeliverable rather than select an invalid or saturated route.

```
graph TD; 1([1. User enters natural-language request]) --> 2([2. Parse and validate origin, destination, payload]); 2 --> 3([3. Generate Phase 1 physics baseline route]); 2 -- "Invalid / ambiguous" --> Fallback([Conservative fallback / undeliverable response]); 3 --> 4([4. Fetch one live /state snapshot for current tick]); 4 -- "API unavailable and cache stale" --> Fallback; 4 --> 5([5. At current node, list safe candidate outgoing links]); 5 -- "no safe link" --> Fallback; 5 --> 6([6. Invoke congestion, trust and targeting tools for each candidate link]); 6 --> 7([7. Compute combined True Cost and uncertainty penalty]); 7 --> 8([8. Re-run shortest-path from current node using adjusted costs]); 8 --> 9([9. Commit first hop, transmit, append audit record]); 9 --> 10{Destination reached?}; 10 -- "yes" --> 10a([10. Produce exact unified JSON schema + explanation]); 10 -- "no - next node" --> 4;
```

7.2 Important interpretation

7.3 Safety guards

- Page 16

8. Proposed Code Structure

```

AlchemiX/
├── app.go
├── main.go
├── universe-config.json
├── .env.example
├── docker-compose.yml
├── internal/
│   ├── agent/
│   │   ├── orchestrator.go
│   │   ├── decision.go
│   │   ├── true_cost.go
│   │   └── fallback.go
│   ├── nlparse/
│   │   ├── parser.go
│   │   └── parser_test.go
│   ├── telemetry/
│   │   ├── client.go
│   │   ├── types.go
│   │   └── cache.go
│   ├── analytics/
│   │   ├── client.go
│   │   └── types.go
│   ├── audit/
│   │   ├── logger.go
│   │   └── explanation.go
│   ├── routing/           # existing Phase 1 logic
│   ├── latency/           # existing Phase 1 logic
│   ├── resilience/        # existing Phase 1 logic
│   ├── packet/            # existing Phase 1 logic
│   └── service/            # existing transmission service
├── analytics/
├── pyproject.toml
├── Dockerfile
├── app/
│   ├── main.py
│   ├── schemas.py
│   ├── model_loader.py
│   └── tools/
│       ├── congestion.py
│       ├── trust.py
│       └── targeting.py
├── training/
│   ├── validate_data.py
│   ├── train_congestion.py
│   ├── train_trust.py
│   ├── train_targeting.py
│   └── evaluate_all.py
├── models/
│   ├── congestion.joblib
│   ├── trust.json
│   ├── targeting.joblib
│   └── model_manifest.json
├── tests/
├── frontend/src/
│   ├── components/
│   │   ├── AgentRequest.tsx
│   │   ├── RouteComparison.tsx
│   │   ├── LinkEvaluationCard.tsx
│   │   └── AgentAuditTimeline.tsx
│   └── types/
├── docs/
│   ├── phase2-architecture.md
│   ├── model-card.md
│   └── demo-runbook.md

```

9. Environment Setup and Installation Order

9.1 Prerequisites

Install or verify the following before changing source code:

18. Git.
19. Go compatible with the existing `go.mod`.
20. Node.js compatible with the existing frontend lock file.
21. Wails v2 CLI.
22. A C/C++ compiler required by Wails for the host operating system.
23. Python 3.11 or newer.
24. A Python virtual-environment tool (`venv` is sufficient).
25. Docker Desktop or Docker Engine with Docker Compose v2.
26. Optional: Make for repeatable commands.

Do not upgrade Go, Node, Wails, or frontend dependencies until the current project builds successfully. The repository's lock files and module files are the source of truth.

9.2 First commands after repository access

```
# 1. Create a Phase 2 branch
git checkout -b feature/phase2-analytical-copilot

# 2. Confirm the current Go application builds
go mod download
go test ./...

# 3. Confirm the frontend builds
cd frontend
npm ci
npm run build
cd ..

# 4. Confirm Wails development mode
wails dev

# 5. Confirm production build
wails build

# 6. Inspect current Docker setup
docker compose config
docker compose up --build
```

If any baseline command fails, record and fix it before Phase 2 work. Do not mix Phase 1 repair with agent features in the same commit.

9.3 Create the analytics environment

```
cd analytics
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS/Linux
source .venv/bin/activate

python -m pip install --upgrade pip
```

Initial Python dependencies should include:

```
pandas
numpy
scikit-learn
fastapi
uvicorn
pydantic
joblib
httpx
pytest
```

Pin exact versions only after the environment works and tests pass. Then commit the lock or requirements file.

9.4 Environment variables

Create `.env.example` without secrets:

```
CHIMERA_BASE_URL=https://chimera.launch26.space
CHIMERA_TEAM_KEY=
ANALYTICS_BASE_URL=http://127.0.0.1:8091
TELEMETRY_POLL_INTERVAL_MS=2000
TELEMETRY_STALE_AFTER_MS=10000
MODEL_TIMEOUT_MS=1000
TRUST_BUDGET_MS=50000
TARGET_BUDGET_MS=50000
ENTROPY_BUDGET_MS=25000
UNCERTAINTY_BUDGET_MS=75000
AUDIT_LOG_PATH=./runtime/agent-decisions.jsonl
```

The real key must be loaded from a local `.env` or operating-system secret store and excluded from Git.

10. Implementation Roadmap

Phase 0 - Repository Baseline and Gap Confirmation

Goal: Prove the existing system works before extension.

Tasks:

- Inspect the actual repository tree.
- Run Go tests, frontend build, Wails dev/build, and Docker Compose.
- Capture one real `StartTransmission` result.
- Align TypeScript types with Go structs.
- Confirm existing router interfaces and latency fields.
- Confirm Docker service names and ports.

Deliverables:

- Baseline build report.
- List of defects separated from Phase 2 work.
- Confirmed integration points.

Exit criteria:

- Phase 1 transmission works end to end.
- A known input produces a repeatable baseline route and hop log.

Phase 1 - Data Validation and Reproducible Training Foundation

Goal: Make the supplied CSVs trustworthy and reproducible.

Tasks:

- Add schema validation for all columns and types.
- Validate 12 links against `universe-config.json`.
- Normalize alphabetical link IDs.
- Report missing data and invalid ranges.
- Create time-based train/validation/test splits.
- Produce a dataset profile and hashes.

Deliverables:

- Data-validation script.
- Clean training views, not modified source files.
- Model manifest template.

Exit criteria:

- Training fails loudly on schema drift.
- No live `/state` values are used for final model training.

Phase 2 - Train the Three Analytical Models

Goal: Produce versioned, measurable model artifacts.

Tasks:

- Train congestion benchmark and champion models.
- Derive trust priors and live plausibility logic.
- Train and calibrate targeting-risk model.
- Evaluate time-based metrics.
- Define out-of-distribution rules.
- Write model cards and decision explanations.

Deliverables:

- Model artifacts.
- Model manifest with metrics and features.
- Reproducible training commands.

Exit criteria:

- Every model has a clear input/output contract.
- The selected model outperforms its simple baseline.
- Trust findings are explainable link by link.

Phase 3 - Analytical Tool Service

Goal: Expose stable local tool calls.

Tasks:

- Implement FastAPI schemas.
- Load models once at startup.
- Add `/health` and `/model-info`.
- Add prediction endpoints.
- Add timeouts, input validation, and structured logs.
- Containerize the analytics service.

Deliverables:

- Running sidecar.
- OpenAPI contract.
- Unit and API tests.

Exit criteria:

- Invalid input returns a clear 4xx response.
- Valid predictions complete within the selected latency budget.
- Health check reports loaded model versions.

Phase 4 - Live Telemetry Integration

Goal: Consume current network state safely.

Tasks:

- Implement Go telemetry client.
- Add header authentication from environment.
- Validate `/links` and `/state` responses.
- Add cache, timeout, bounded retries, and stale-state rules.
- Treat saturated links as unavailable.
- Add a fake API server for tests.

Deliverables:

- Typed telemetry package.
- Integration tests without depending on the real API.

Exit criteria:

- 401, timeout, malformed JSON, null latency, and saturation are tested.
- The real API key never appears in logs or source.

Phase 5 - Natural-Language Request Parser

Goal: Meet the unstructured input requirement deterministically.

Tasks:

- Build a planet-name lexicon from config.
- Support common request patterns.
- Extract quoted and unquoted payloads.
- Detect missing, duplicate, same-origin, and ambiguous planets.
- Return parsed values with confidence and error messages.

Deliverables:

- Parser package and test corpus.
- UI preview of parsed fields.

Exit criteria:

- Parser passes normal, mixed-case, punctuation, and ambiguous-input tests.
- It never silently invents a planet or payload.

Phase 6 - Sequential Co-Pilot and True-Cost Routing

Goal: Implement the core Phase 2 behavior.

Tasks:

- Add the per-hop agent state machine.
- Generate the Phase 1 baseline route.
- Evaluate baseline and alternative outgoing links.
- Call all three tools.
- Calculate True Cost and uncertainty.
- Re-run the shortest-path algorithm with adjusted weights.
- Commit one hop and repeat.
- Add loop, hop-count, and no-route guards.
- Implement fallback behavior when a tool is unavailable.

Deliverables:

- Agent package.
- Exact unified response object.
- Detailed internal audit objects.

Exit criteria:

- A live link change can alter the next hop.
- Every selected link has three model outputs and a combined cost.
- L_{max} and failure rules remain enforced.

Phase 7 - UI and Explainability

Goal: Make the agent visible and defensible.

Tasks:

- Add natural-language request panel.
- Show parsed request preview.
- Show baseline vs chosen route.
- Add per-link evaluation cards.
- Add current tick and API health.
- Show why a link was rejected or selected.
- Show uncertainty and fallback warnings.
- Preserve current conversion and hop-log views.

Deliverables:

- Updated React components and types.
- Screenshot-ready demo state.

Exit criteria:

- A judge can follow the decision without reading code.
- No required output field is hidden or renamed.

Phase 8 - Resilience, Chaos, and Generalization Testing

Goal: Prepare for the live trials.

Tasks:

- Sever the most used link during an active route.
- Simulate analytics-service failure.
- Simulate API timeout, 401, stale tick, malformed response, and null fields.
- Inject unseen load ratios and traffic shares.
- Verify no packet loss, crash, infinite loop, or L_{max} violation.
- Verify fallback and uncertainty explanations.

Deliverables:

- Automated chaos test suite.
- Demo evidence and logs.

Exit criteria:

- The packet either reroutes safely or returns a controlled undeliverable result.
- The application remains responsive.

Phase 9 - Packaging and Final Runbook

Goal: Make the system repeatable on the judging machine.

Tasks:

- Pin dependencies.
- Finalize Docker Compose and health checks.
- Add one-command startup and shutdown.
- Run Go, Python, frontend, Wails, and Compose tests.
- Produce README, architecture, model card, and demo script.
- Remove secrets and unused code.

Deliverables:

- Final build.
- Demo runbook.
- Decision-audit examples.

Exit criteria:

- A clean machine can start the system using documented commands.
- Output matches the challenge schema exactly.

11. Testing Strategy

11.1 Unit tests

- Natural-language parsing.
- Alphabetical link-ID normalization.
- True-Cost calculations.
- Saturated-link handling.
- Confidence and uncertainty conversion.
- Route-reuse decay.
- Exact output serialization.

11.2 Model tests

- Schema validation.
- Time-based performance metrics.
- Reproducibility with fixed random seeds.
- Feature-range and missing-value behavior.
- Probability bounds from 0 to 1.
- Model artifact and manifest version match.

11.3 Integration tests

- Go agent to FastAPI tool calls.
- Go telemetry client to fake Chimera API.
- Wails method to React state update.
- Baseline router to adjusted router.
- Decision log to UI event stream.

11.4 Acceptance scenarios

27. **Normal delivery:** all links healthy; chosen route may equal baseline.
28. **Congested baseline link:** agent chooses a lower True-Cost detour.
29. **Spoofed link:** low trust raises cost despite attractive self-reported latency.
30. **High traffic-share link:** targeting risk and route-reuse penalty promote diversity.
31. **Saturated link:** link is removed immediately.
32. **Live severing:** current route is recomputed at the next safe decision point.
33. **Unseen vector:** uncertainty is flagged and conservative fallback is used.

34. **No valid route:** controlled undeliverable result with explanation.
35. **Analytics unavailable:** safe fallback route or controlled failure, never a crash.
36. **Malformed natural language:** parser asks for correction rather than guessing.

12. Non-Functional Requirements

Category	Requirement
Reliability	No unhandled exception during telemetry, model, or rerouting failure
Performance	Per-hop decision should fit within a defined budget; model service loaded once
Security	Team key only in environment; no secret in UI, logs, code, or Git
Explainability	Every link score has input values, model version, penalty components, and reason
Maintainability	Existing Phase 1 packages remain isolated from new agent packages
Testability	Real API is replaceable with a fake server; model service has deterministic fixtures
Portability	Documented startup on the team's judging operating system
Observability	Correlation ID across UI, Go logs, Python logs, and audit records
Compliance	Exact output schema and alphabetical link naming

13. Risks and Mitigations

Risk	Impact	Mitigation
Actual repository differs from handoff	Integration delays	Start with Phase 0 repository audit and build proof
Very few saturated examples	Poor failure classifier	Use live status and ≥ 0.90 guard; do not rely on learned classifier
Model predicts outside training range	Unsafe route	Add OOD detection and uncertainty penalty
FastAPI sidecar fails	Agent cannot score links	Health check, timeout, conservative fallback, cached priors
API values are scrambled during build days	Wrong model assumptions	Train only from CSVs; use live API only for contract tests
API key leaks	Submission/security issue	Environment variables, redaction, <code>.gitignore</code> , log filters
Static optimal path becomes predictable	Higher jam risk	Traffic-share model plus decayed route-reuse penalty
Purely greedy routing reaches a dead end	Failed delivery	Re-run shortest path globally from current node using adjusted costs
Frontend duplicates routing logic	Inconsistent results	Keep Go backend as source of truth
Explanation does not match model	Audit failure	Generate explanation from the same stored penalty components
Dependency upgrades break Wails	Build failure	Build existing lock files first; upgrade only with a separate decision

14. Requirements Traceability Matrix

Challenge requirement	Proposed component	Verification
Parse natural-language request	<code>internal/nlparse</code>	Parser unit-test corpus
Preserve baseline physics route	Existing <code>internal/routing</code>	Phase 1 regression tests
Sequential node-by-node agent	<code>internal/agent/orchestrator.go</code>	Multi-hop integration test
Congestion prediction	Python congestion tool	Time-based MAE/RMSE and API test
Trust score	Python trust tool	Known spoofed-link evidence and boundary tests
Targeting risk	Python targeting tool	ROC/AP/Brier and traffic-share tests
Dynamic True Cost	Go True-Cost engine	Exact numeric unit tests
Reroute on severed link	Agent plus resilience/router	Chaos test
Handle unseen event	OOD and fallback module	Injected extreme-value tests
Exact output schema	Go response DTO	JSON snapshot/schema test
Alphabetical link ID	Shared normalizer	Unit tests for both orders
Decision explanation	Audit/explanation builder	Golden-file tests and live walkthrough
Poll live state reasonably	Telemetry cache and interval	Integration timing test
No training on build-day state	Training pipeline boundary	Data-source manifest audit

15. Definition of Done

Phase 2 is complete only when:

- The existing Phase 1 route and transmission tests still pass.
- A natural-language request is parsed into origin, destination, and payload.
- The agent evaluates congestion, trust, and targeting risk at every hop.
- Saturated, failed, and over-distance links are never selected.
- The route can change after a live state change.
- The exact required JSON structure is produced.
- Every `link_evaluations` record can be explained from stored inputs and penalty components.
- The unseen-vector test activates a safe, explicit fallback.
- The team key is not present in the repository or logs.
- Go tests, Python tests, frontend build, Wails build, and Docker Compose checks pass.
- A clean-start demo runbook has been tested.

16. Recommended First Work Session After Release

Once the exact release phrase is received, implementation should begin in this order:

37. Inspect and build the real repository without modifications.
38. Capture one real Phase 1 transmission result and Go structs.
39. Create the Phase 2 branch and baseline test evidence.
40. Add the analytics folder and data-validation script.
41. Reproduce the preliminary dataset findings.
42. Train and compare the three analytical components.
43. Expose the local tool API.
44. Integrate the telemetry client and deterministic parser.
45. Build the sequential agent and True-Cost engine.
46. Update the UI, then execute chaos and packaging work.

No implementation activity has been performed as part of this report.

17. Source Material

This blueprint is based on:

- `PROJECT_HANDOFF_README.md`
- `Launch26 - Phase 02 - Challenge.pdf`
- `DATASETS.md`
- `universe-config.json`
- `link_traffic_history.csv`
- `link_telemetry.csv`
- `link_incident_history.csv`

Release Gate

The planning stage is complete. The next stage is intentionally blocked until the user sends the exact phrase:

RELEASE THE KRAKEN